In [1]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Connect to SQLite database
conn = sqlite3.connect("C:/Users/ankit/Downloads/New Folder (3)/Task 7/sales_data.db")
```

In [2]:
```
import sqlite3
import pandas as pd

# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Use full path if needed

# SQL query to select all data
query_all = "SELECT * FROM sales"

# Load data into a DataFrame
df_all = pd.read_sql_query(query_all, conn)

# Close the connection
conn.close()

# Display the DataFrame
print("All Sales Data:")
print(df_all)
```

```
All Sales Data:
   id product  quantity  price
0   1   Apples        10    2.5
1   2  Bananas        15    1.2
2   3  Oranges        20    1.5
3   4   Apples         5    2.5
4   5  Bananas        10    1.2
```

In [3]:
```
import sqlite3
import pandas as pd

# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Adjust path if needed

# SQL query to calculate total revenue
query_revenue = "SELECT SUM(quantity * price) AS total_revenue FROM sales"

# Execute the query and load the result into a DataFrame
df_revenue = pd.read_sql_query(query_revenue, conn)

# Close the connection
conn.close()

# Display the total revenue
print("Total Revenue:")
print(df_revenue)
```

```
Total Revenue:
   total_revenue
0          99.0
```

In [4]:
```
import sqlite3
import pandas as pd

# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Adjust path if needed

# SQL query to calculate total quantity sold per product
query_quantity = """
SELECT
    product,
    SUM(quantity) AS total_quantity
FROM sales
GROUP BY product
"""

# Execute the query and load the result into a DataFrame
df_quantity = pd.read_sql_query(query_quantity, conn)

# Close the connection
conn.close()

# Display the total quantity sold per product
print("Total Quantity Sold per Product:")
print(df_quantity)
```

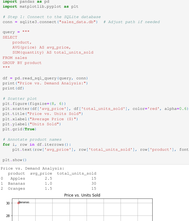
```
Total Quantity Sold per Product:
   product  total_quantity
0   Apples              15
1  Bananas              25
2  Oranges              20
```

In [5]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")

# SQL query to calculate total quantity sold per product
query_quantity = """
SELECT
    product,
    SUM(quantity) AS total_quantity
FROM sales
GROUP BY product
"""

# Execute the query and load the result into a DataFrame
df_quantity = pd.read_sql_query(query_quantity, conn)

# Close the connection
conn.close()

# Print the results
print("Total Quantity Sold per Product:")
print(df_quantity)

# Plot the bar chart
plt.figure(figsize=(8, 5))
plt.bar(df_quantity["product"], df_quantity["total_quantity"], color="skyblue")
plt.title("Total Quantity Sold per Product")
plt.xlabel("Product")
plt.ylabel("Total Quantity")
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.savefig("quantity_per_product.png")  # Optional: Save the chart
plt.show()
```

```
Total Quantity Sold per Product:
   product  total_quantity
0   Apples              15
1  Bananas              25
2  Oranges              20
```

In [6]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Adjust path if needed

# Step 2: Run SQL query to get total revenue per product
query_revenue = """
SELECT
    product,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""

# Step 3: Load query results into a DataFrame
df_revenue = pd.read_sql_query(query_revenue, conn)

# Step 4: Close the connection
conn.close()

# Step 5: Print the DataFrame
print("Revenue per Product:")
print(df_revenue)

# Step 6: Plot the revenue bar chart
plt.figure(figsize=(8, 5))
plt.bar(df_revenue["product"], df_revenue["revenue"], legend=False, color="coral")
plt.title("Revenue per Product")
plt.xlabel("Product")
plt.ylabel("Revenue")
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig("revenue_per_product.png")  # Optional: Save chart
plt.show()
```

```
Revenue per Product:
   product  revenue
0   Apples     37.5
1  Bananas     30.0
2  Oranges     33.0
figure size 720x432 with 0 Axes
```



In [7]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Adjust path if needed

# Step 2: SQL query to calculate average units sold per product
query = """
SELECT
    product,
    AVG(quantity) AS avg_units_sold
FROM sales
GROUP BY product
"""

# Step 3: Load the result into a DataFrame
df_avg_units = pd.read_sql_query(query, conn)

# Step 4: Close the database connection
conn.close()

# Step 5: Print the average units sold
print("Average Units Sold per Product:")
print(df_avg_units)

# Step 6: Plot the bar chart
plt.figure(figsize=(8, 5))
plt.bar(df_avg_units["product"], df_avg_units["avg_units_sold"], legend=False, color="mediumpurple")
plt.title("Average Units Sold per Product")
plt.xlabel("Product")
plt.ylabel("Average Units Sold")
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig("avg_units_sold.png")  # Optional: Save chart
plt.show()
```

```
Average Units Sold per Product:
   product  avg_units_sold
0   Apples            7.5
1  Bananas           12.5
2  Oranges           15.0
figure size 720x432 with 0 Axes
```

In [8]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Adjust path if needed

query = """
SELECT
    product,
    SUM(quantity) AS total_units_sold
FROM sales
GROUP BY product
ORDER BY total_units_sold DESC
LIMIT 5
"""

df = pd.read_sql_query(query, conn)
print("Top 5 Products by Units Sold:")
print(df)

plt.barh(df["product"], df["total_units_sold"], color="teal", title="Top-Selling Products (Units)")
plt.ylabel("Units Sold")
plt.show()
```

```
Top 5 Products by Units Sold:
   product  total_units_sold
0  Bananas                25
1  Oranges                20
2   Apples                15
```


In [9]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Adjust path if needed

query = """
SELECT
    product,
    AVG(price) AS avg_price,
    SUM(quantity) AS total_units_sold
FROM sales
GROUP BY product
"""

df = pd.read_sql_query(query, conn)
print("Price vs. Demand Analysis:")
print(df)

# Scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["avg_price"], df["total_units_sold"], color="red", alpha=0.5)
plt.title("Price vs Units Sold")
plt.xlabel("Average Price ($)")
plt.ylabel("Units Sold")
plt.grid(True)

# Annotate product names
for i, row in df.iterrows():
    plt.text(row["avg_price"], row["total_units_sold"], row["product"], fontsize=9)

plt.show()
```

```
Price vs. Demand Analysis:
   product  avg_price  total_units_sold
0   Apples        2.5                15
1  Bananas        1.2                25
2  Oranges        1.5                20
```


In [10]:
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import PercentFormatter

# Step 1: Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")  # Make sure the database exists

# Step 2: Execute transaction count query
query = """
SELECT
    product,
    COUNT(*) AS num_transactions,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM sales), 1) AS pct_of_total
FROM sales
GROUP BY product
ORDER BY num_transactions DESC
"""

df = pd.read_sql_query(query, conn)

# Step 3: Close the database connection
conn.close()

# Step 4: Print formatted results
print("Transaction Count by Product:")
print(df.to_string(index=False))

# Step 5: Enhanced visualization
plt.figure(figsize=(12, 6))

# Bar plot with percentage annotations
bars = plt.bar(df['product'], df['num_transactions'], color='#1f77b4', alpha=0.7)
plt.title('Product Transaction Frequency', pad=20, fontsize=14)
plt.xlabel('Product', labelpad=10)
plt.ylabel('Number of Transactions', labelpad=10)
plt.setp(plt.xticks()[1], rotation=0, ha='right')

# Add percentage labels on top of bars
for bar, pct in zip(bars, df['pct_of_total']):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{pct:.1f}%', ha='center', va='bottom')

# Add horizontal grid lines
plt.gca().yaxis.set_major_formatter(PercentFormatter(xmax=1, decimals=0))

# Adjust layout to prevent label cutoff
plt.tight_layout()

# Save and display
plt.savefig('transaction_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

# Bonus: Create a table-style summary
fig, ax = plt.subplots(figsize=(8, 3))
ax.axis('off')
table = ax.table(cellText=df.values,
                 colLabels=df.columns,
                 loc='center',
                 cellLoc='center')
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1.2, 1.5)
plt.title('Transaction Summary', y=0.8, fontsize=14)
plt.savefig('transaction_table.png', dpi=300, bbox_inches='tight')
```

```
Transaction Count by Product:
 product  num_transactions  pct_of_total
Bananas                 2          40.0
 Apples                 2          40.0
Oranges                 1          20.0
```


In [11]:
```

```